



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

MDE

MODELAÇÃO EM FRAMES – PROLOG

LAB 3

- ☐ Labwork 3 classes planning
- ☐ Some revisions
- ☐ Lab Work Requirements
- ☐ Air Conditioning example - Implementation

➤ **Class 1 [18.05.2026 – 22.05.2026]:**

- Presentation of the work assignment and the tools to be used.
- Review of concepts related to programming using Prolog with the Golog library.
- Presentation of the problem.
- Implementation of the AC control system.

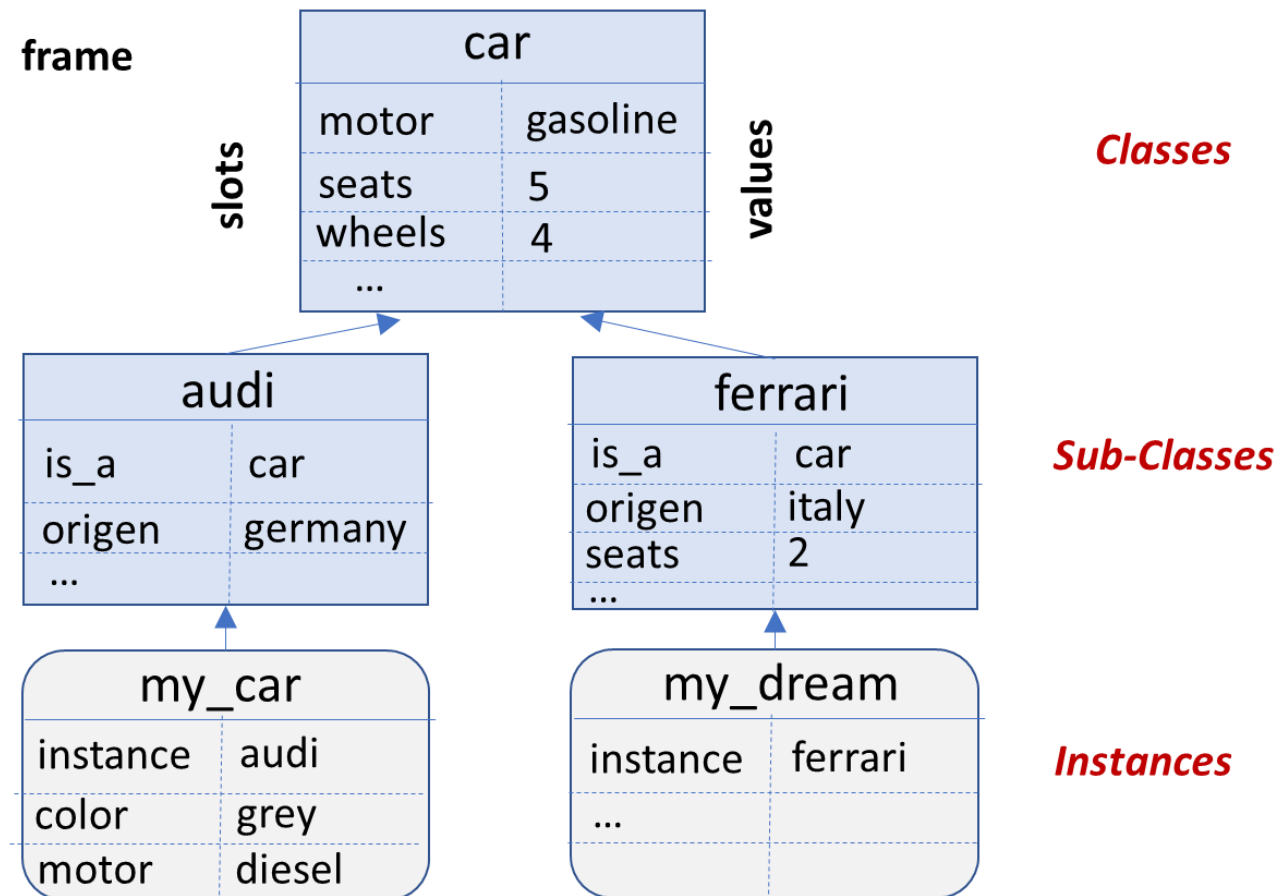
➤ **Class 2 [25.05.2026 – 29.05.2026]:**

- Implementation of the functional requirements for the **Smart Warehouse** management system.
- UML diagrams required in part II.
- Finalization of the assignment.

“FRAMES” TAXONOMY



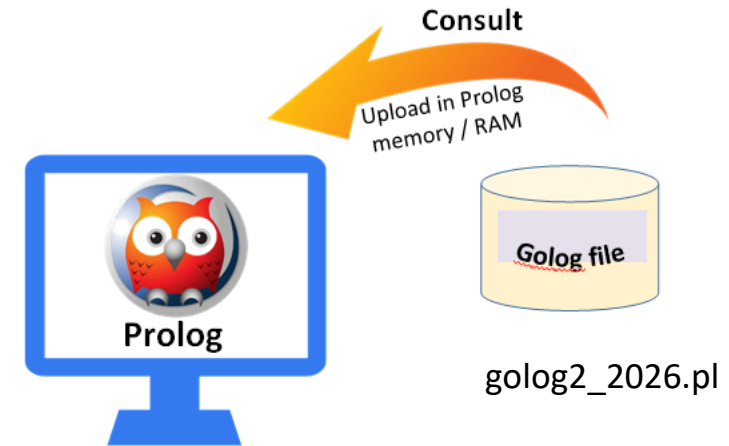
Frames.
Classes. Instances.
Slots. Relations.



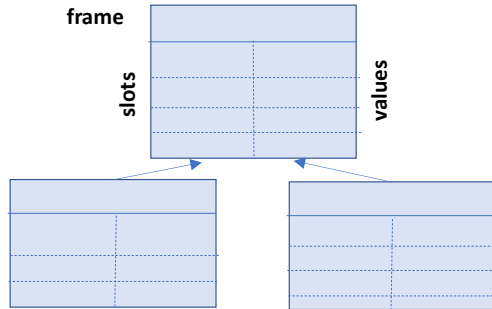


A mini Frame Engine written in Prolog

- Creation and manipulation of frames and their slots and values.
- Definition of relations and inheritance mechanisms.
- Definition of methods and reactive programming.



GOLOG was inspired on a commercial product (KNOWLEDE CRAFT), one of the first frame engines



A. On frames

- new_frame(F)
- frame_exists(F)
- show_frame(F)
- delete_frame(F)

B. On slots

- new_slot(F,S)
- new_slot(F,S,V)
- delete_slot(F,S)
- frame_local_slots(F,LS)
- get_all_slots(F,LS)

C. On values

- new_value(F,S,V)
- new_values(F,S,LV)
- add_value(F,S,V)
- add_values(F,S,LV)
- get_value(F,S,V)
- get_values(F,S,LV)
- delete_value(F,S,V)
- delete_values(F,S)

D. On relations

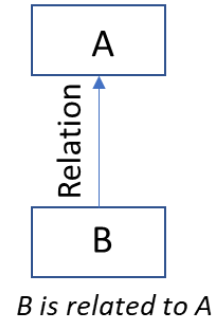
- new_relation(Relation,Transitivity, Restriction, Inverse)
- delete_relation(Relation)
- frame_actual_relations(F,LR)

E. On methods

- new_slot(F, S, Method)
- call_method(F, S, LPar)
- call_method_0(F, S)
- call_method_1(F, S, P)
- call_method_2(F, S, P1, P2)
- call_method_3(F, S, P1, P2, P3)

F. On attached predicates or demons

- new_demon(F,S,Demfunc, Demtype, When, Effect)
- add_demon(F,S,Demfunc, Demtype, When, Effect)
- remove_all_demons(F,S)



D. On relations

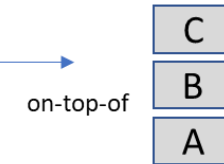
• **new_relation(Relation, Transitivity, Restriction, Inverse)**

*Defines a new relation with name Relation.
The definition requires 4 parameters*

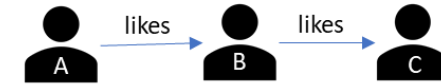
Transitivity: transitive, intransitive

Restriction: all
none
inclusion(LS)
exclusion(LS)

Inverse: name of the inverse relation or *nil*



The relation “on-top-of” is transitive:
If B is on-top-of A and C is on-top-of B, then we can infer that C is on-top-of A



The relation “likes” is intransitive:
If A likes B and B likes C, we cannot infer that A likes C

• **delete_relation(Relation)**

• **frame_actual_relations(F, LR)**

To delete a relation of name Relation

Obtains a list LR with the names of all relations associated to a given frame F

Allows to automatically create an inverse relation. If we don't want it, put “nil” in this parameter

Specifies the level of inheritance:

all – B inherits all slots of A
none – B does not inherit any slot from A
inclusion(LS) – B only inherits from A the slots mentioned in list LS
exclusion(LS) – B inherits all slots from A except those indicated in list LS

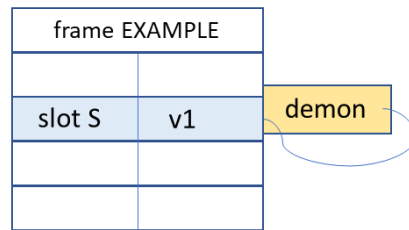
E. On methods

A **method** in frames is a procedure associated with a **class**.
A **method** defines the behavior of the **class** and its sub-classes and instances. A **method** is an action that an object (class) is able to perform.

- **new_slot(F, S, Method)** → *Creates a new Method, which is “stored” in a new slot S.*
- **call_method(F, S, LPar)** → *Calls the method identified by slot S, passing the list of parameters Lpar to the corresponding procedure*
- **call_method_0(F, S)**
 - **call_method_1(F, S, P)**
 - **call_method_2(F, S, P1, P2)**
 - **call_method_3(F, S, P1, P2, P3)** } *Particular cases of call_method for the cases that the corresponding procedure has 0, 1, 2, or 3 parameters, respectively*

call_method_0(F, S) equivalent to call_method(F, S, [])
call_method_1(F, S, P) “ “ call_method(F, S, [P])
call_method_2(F, S, P1, P2) “ “ call_method(F, S, [P1, P2])
call_method_3(F, S, P1, P2, P3) “ “ call_method(F, S, [P1, P2, P3])

F. On attached predicates or demons

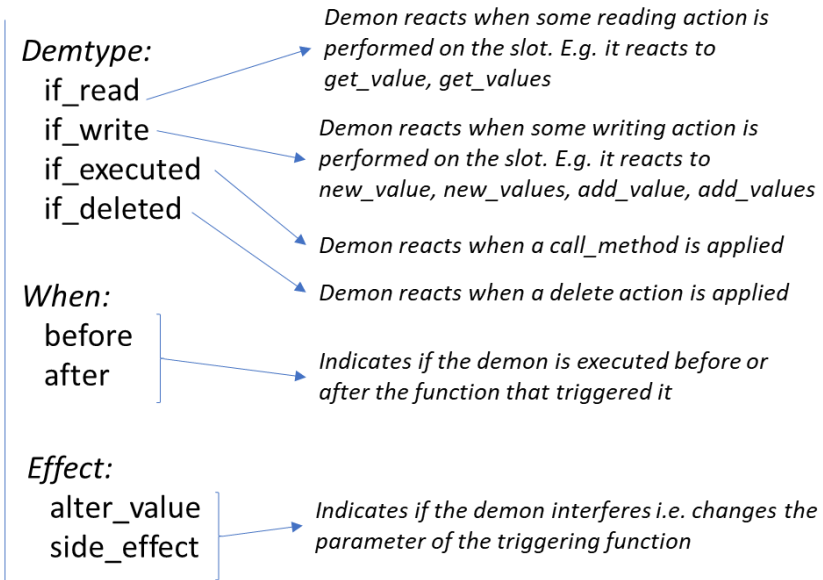


A predicate associated to a slot (hidden) that reacts (executes) when certain actions are performed on the slot

•**add_demon(F,S,Demfunc, Demtype, When, Effect)**

•**remove_all_demons(F,S)**

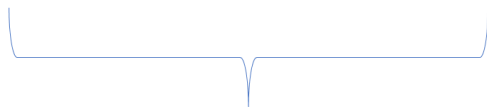
•**new_demon(F,S,Demfunc, Demtype, When, Effect)**



Format of the rule associated to the slot:

E.g.:

transform(Frame, Slot, Received_value, Returned_value) :-



The function (rule) that implements the demon has always 4 parameters

- Mechanism to save the knowledge-base to a file
`?- save_kb(FileName).`
- Mechanism to show the actual knowledge-base
`?- show_kb.`
- Mechanism to delete the knowledge-base
`?- delete_kb.`
- Others...



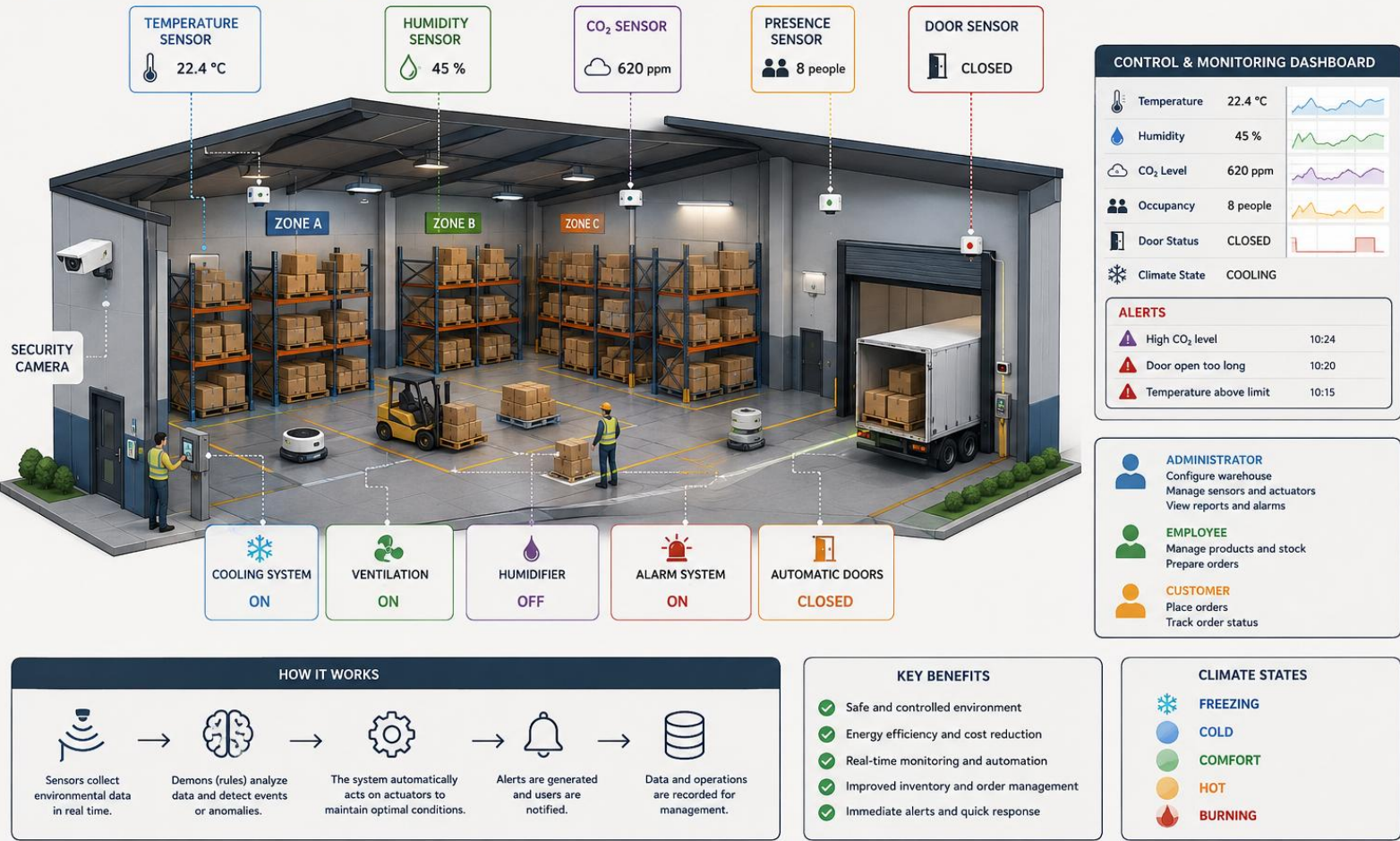
golog2_2026.pl

```
% %%  
% %%   Predicados Utilitarios  
% %%  
% =====  
  
:-style_check(-singleton).  
:-style_check(-discontiguous).  
  
:-dynamic frame_/1.  
:-dynamic relation_/1.  
  
% http://stackoverflow.com/questions/130097/real-world-prolog-usage  
%  
%  
  
save_kb(Filename):-  
    tell(Filename),  
    forall(show_kb,true),  
    told.  
  
show_kb:-  
    ( ( clause(frame_(_),true),  
      listing(frame_),  
      forall(frame_(Frame),listing(Frame)) )
```

LABWORK 3

SMART WAREHOUSE

Intelligent Monitoring, Automation and Management



AI-generated image created with ChatGPT (OpenAI), 2026, adapted by the authors.

Ficha de trabalho Nº 3	
Disciplina	Modelação de Dados em Engenharia
Ano Lectivo	2025/2026
Objectivo	Modelar conhecimento usando Frames e UML
Aulas	2 aulas x 3 horas + 3 horas extra
Data de Entrega	2026/06/02

Objetivos concretos:

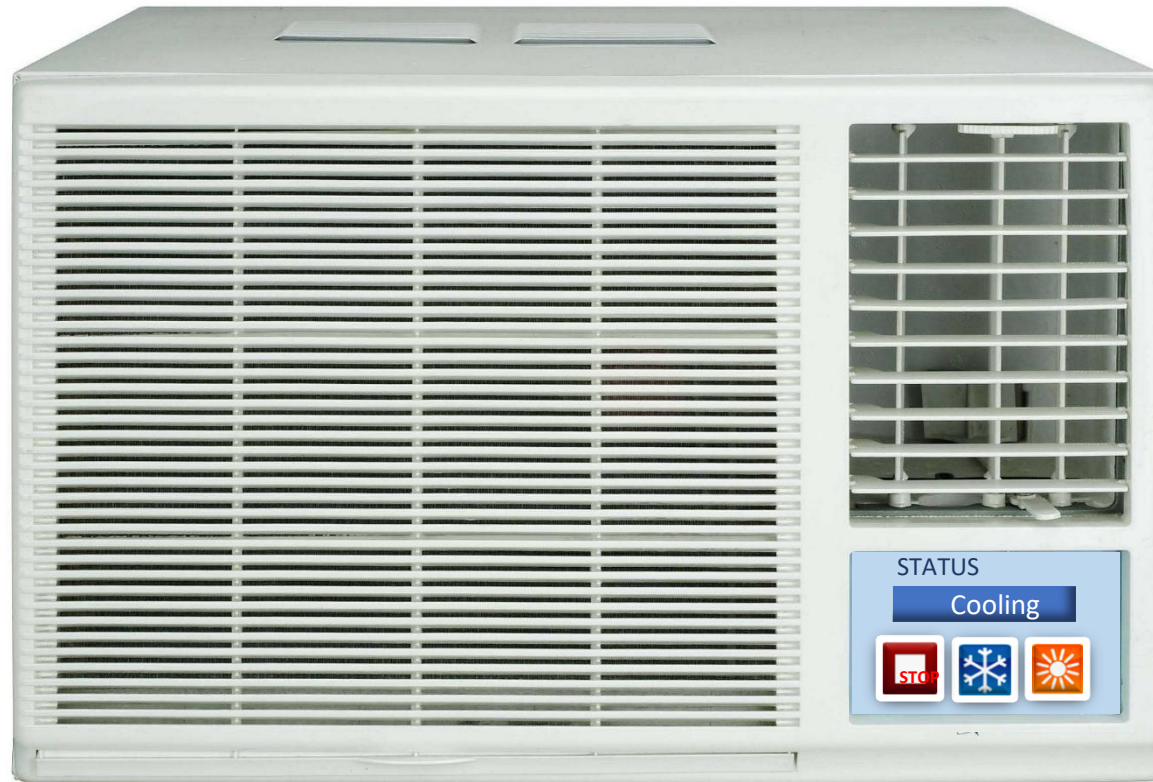
1. Modelação baseada em frames
- a. Entender o conceito de modelação com frames e relevância num contexto de engenharia Electrotécnica.
 - b. Representação de modelos com a linguagem Prolog usando a biblioteca fornecida GOLOG.
 - c. Modelar informação utilizando conceitos de: frames, slots, relações, mecanismos de herança, métodos e demons.

Req. Funcionais	Descrição
FR1	Criar, visualizar, alterar e apagar armazéns.
FR2	Criar e gerir sensores e atuadores (incluindo simulação de medições e atualizações em tempo real).
FR3	Criar, visualizar e atualizar produtos e stock.
FR4	Criar encomendas considerando disponibilidade.
FR5	Visualizar estados das encomendas.
FR6	Configurar e simular climatização automática.
FR7	Implementar adaptação automática do sistema de climatização baseada em ocupação.
FR8	Gerar e visualizar mensagens de alarme.
FR9	Diagrama de Classes que suporta o problema
FR10	Diagrama de Sequência ilustrando interação entre leitura de sensores; ativação de demons; atuação dos atuadores; e geração de alarmes.

EXAMPLE

Model an air conditioning system

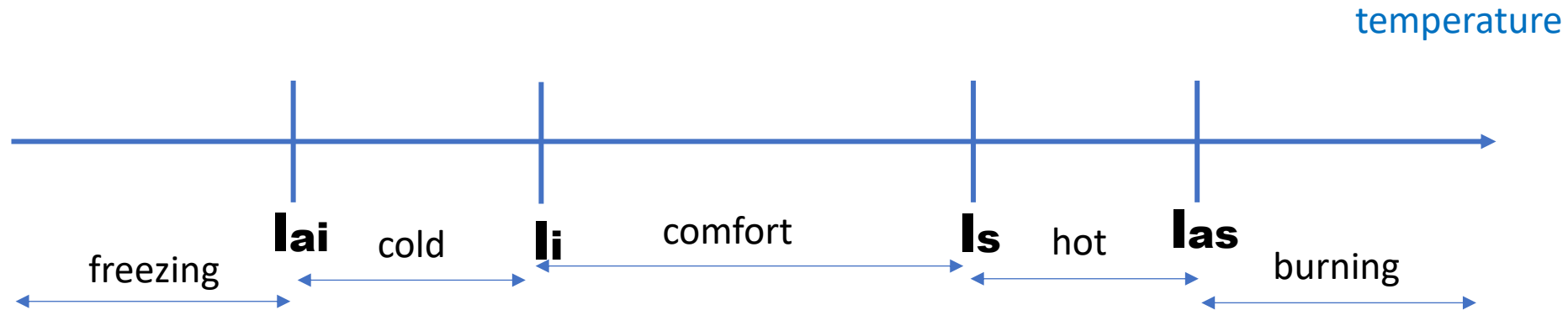
AC



Use this example on your Labwork 3



The scale of temperatures is characterized by the following diagram:

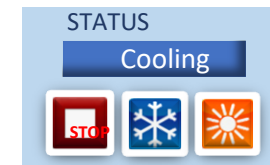


The level of comfort / discomfort in the room (“**clima**”) is classified in 5 levels:

freezing, cold, comfort, hot, burning

Possible **status** of the AC: inactive, warming, cooling

AC offers 3 **operations**: stop, warm, cool



When we check the **temperature** in AC it automatically gets it from the Thermostat

The **control** system is automatic and depends on the temperature in the Thermostat

When the **absolute** upper or lower **limits** L_{as} or L_{ai} are passed, the controller generates an alarm message.

An **alarm** is characterized by:

- event:
- temp: temperature recorded on that occasion
- date: date and time of the alarm event

Generated alarms (represented by frames) are named as alarm1, alarm2, alarm3, ...

```
model_ac:- def_ac, def_thermo, def_alarm.
```


EXAMPLE: Main “Frames”



ac	
status	inactive
temp	
stop	stopf
warm	warmf
cool	coolf

Demon that reacts to read actions and gets the temperature from the Thermostat

readtd

Operations

thermo	
temp	0
li	14
lai	0
ls	25
las	35
clima	

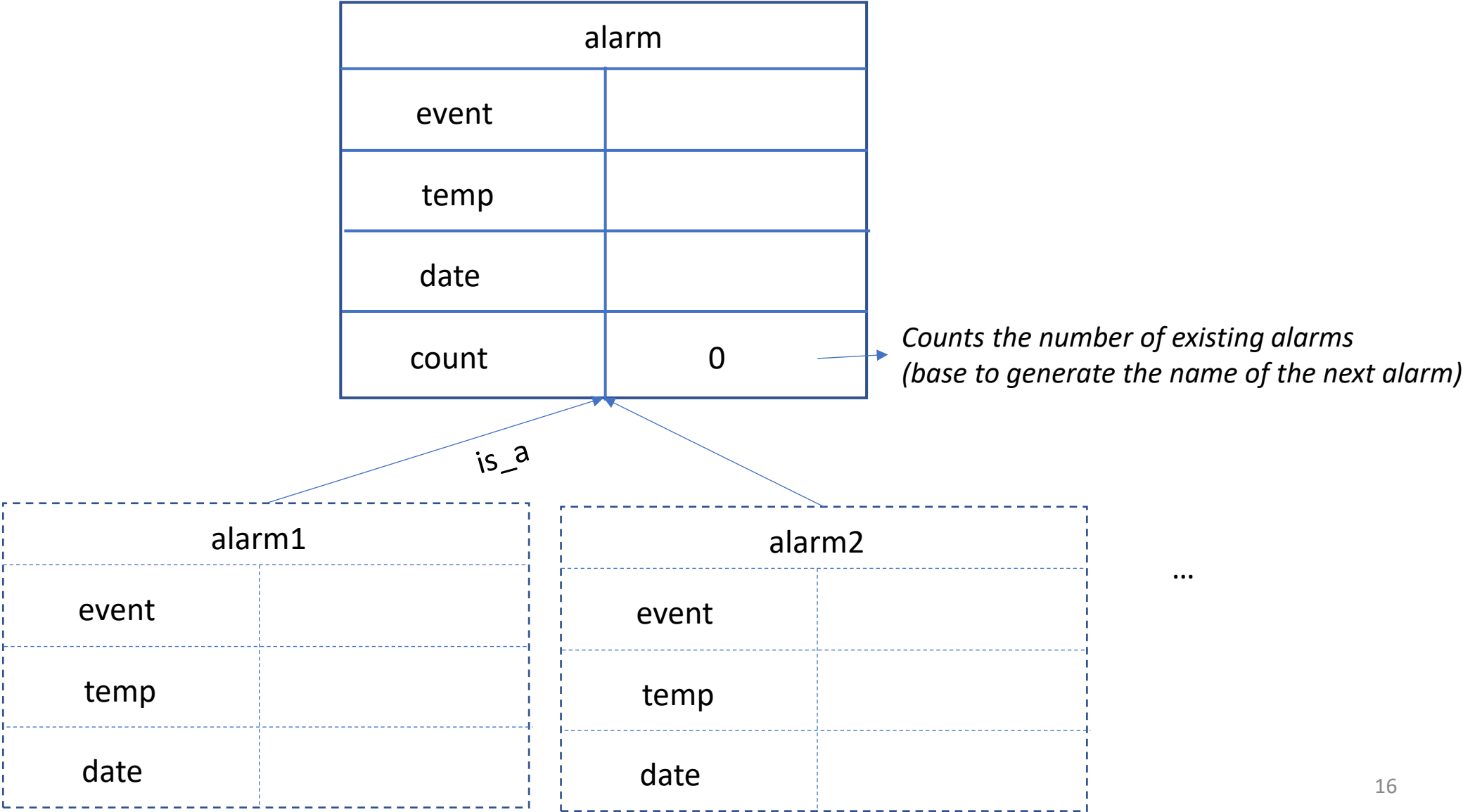
Demon that reacts to write actions and activates the controller

controld

Demon that generates the value of “clima” as a function of the temperature

climad

EXAMPLE: “Frame” alarm



ac	
status	inactive
temp	
stop	stopf
warm	warmf
cool	coolf

readtd

methods

% AIR CONDITIONING

```
def_ac :- new_frame(ac), new_slot(ac, status, inactive),
          new_slot(ac,temp), new_slot(ac,stop,stopf),
          new_slot(ac,warm,warmf),
          new_slot(ac,cool,coolf), def_readtd.
```

```
stopf(F) :- new_value(F, status, inactive).
warmf(F) :- new_value(F, status, warming).
coolf(F) :- new_value(F, status, cooling).
```

```
def_readtd :- new_demon(ac, temp, readtd, if_read, after,
                        alter_value).
```

```
readtd(F, S, _, T) :- get_value(thermo, temp, T).
```

EXAMPLE: Details THERMO

thermo	
temp	0
li	14
lai	0
ls	25
las	35
clima	

controld

climad



% THERMOSTAT

```
def_thermo:- (...),  
             def_climad, def_controlld.
```

Complete!

```
def_climad:-new_demon(thermo, clima, climad, if_read, after,  
                     alter_value).
```

```
climad(F,S,_,C) :- get_value(F, temp,T), get_value(F,li,Li),  
                  get_value(F,lai,Lai), get_value(F,ls,Ls),  
                  get_value(F,las,Las), classify(T,Lai,Li,Ls,Las,C).
```

```
classify(T,_,Li,Ls,_,comfort) :- T>=Li, T<=Ls.
```

```
classify(T,Lai,_,_,freezing) :- T<Lai.
```

```
classify(T,_,_,Las,burning) :- T>Las.
```

```
classify(T,Lai,Li,_,_,cold) :- T>=Lai, T<Li.
```

```
classify(T,_,_,Ls,Las,hot) :- T>=Ls, T<=Las.
```

% CONTROL

```
def_controld :- new_demon(thermo, temp, controld, if_write, before, side_effect).
```

```
controld(F,S,T,T) :- get_value(F,temp,Ta), get_value(F,li,Li), get_value(F,lai,Lai), get_value(F,ls,Ls),  
                    get_value(F,las,Las), act(T,Ta,Li,Lai,Ls,Las).
```

```
act(T,Ta,Li,Lai,Ls,Las) :- T>Ls, call_method_0(ac,cool), malarm(T,Ta,Li,Lai,Ls,Las).
```

```
act(T,Ta,Li,Lai,Ls,Las) :- T<Li, call_method_0(ac,warm), malarm(T,Ta,Li,Lai,Ls,Las).
```

```
act(_,_,_,_,_):-call_method_0(ac,stop).
```

```
malarm(T,Ta,_,_,_Las) :- T>Las, Ta<Las, getdate(D), genmsg(T, burning,D).
```

```
malarm(T,Ta,_Lai,_,_ ) :- T<Lai, Ta>Lai, getdate(D), genmsg(T,freezing,D).
```

```
malarm(_,_,_,_,_).
```

Generate an alarm each time a temperature goes beyond Las or Lai.



```
getdate(D) :- get_time(T), stamp_date_time(T,D,'UTC').
```

Notes:

```
?- get_time(T).
```

```
T = 1684358077.870409
```

A TimeStamp is a floating point number expressing the time in seconds since 1970-01-01

```
?- get_time(T),stamp_date_time(T, D, 'UTC').
```

```
T = 1684358077.870409,
```

```
D = date(2023, 5, 17, 21, 14, 37.870409011, 0, 'UTC', -).
```

Coordinated Universal Time

=> See: <https://www.timeanddate.com/time/zones/>

```
?- get_time(T),stamp_date_time(T, D,local).
```

```
T = 1588683952.532988,
```

```
D = date(2023, 5, 17, 22, 14, 37.87040901184082, -3600, 'GMT Daylight Time', true).
```

year month day hour minutes seconds

*offset relative to
UTC in seconds*

*local
timezone*

*true if daylight saving time
applies to the current time*

EXAMPLE: Details ALARMS



alarm	
event	
temp	
date	
count	0

is_a

alarm1	
event	
temp	
date	

alarm2	
event	
temp	
date	

% ALARMS

def_alarm:- (...),
def_isa.

def_isa:-new_relation(is_a,.....).

genmsg(T, E, D) :- genname(N), new_frame(N),
new_slot(N,is_a,alarm),
new_value(N,event,E),
new_value(N,temp,T),
new_value(N,date,D).

genname(N) :- get_value(alarm,count,A), A1 is A+1,
new_value(alarm,count,A1),
atom_concat(alarm,A1,N).

Complete!

Notes:

atom_concat(*Atom1*, *Atom2*, *Atom3*)

Atom3 forms the concatenation of *Atom1* and *Atom2*.
At least two of the arguments must be instantiated to atoms.

?- atom_concat(alarm,2,A).
A = alarm2.

?- atom_concat(A,2,alarm2).
A = alarm.

?- atom_concat(alarm,N,alarm2).
N = '2'.

← Main program – entry point.

```
model_ac:- def_ac, def_thermo, def_alarm.
```

```
?- model_ac.  
true.
```

```
?- show_frame(ac).
```

```
Frame: ac {  
  temp:  
    Demons: readtd  
  cool: coolf  
  status: inactive  
  stop: stopf  
  warm: warmf  
}
```

```
true.
```

```
?- show_frame(thermo).
```

```
Frame: thermo {  
  clima:  
    Demons: climad  
  lai: 0  
  las: 35  
  li: 14  
  ls: 25  
  temp: 0  
    Demons: controld  
}
```

```
true.
```

```
?- show_frame(alarm).
```

```
Frame: alarm {  
  date:  
  event:  
  temp:  
  count: 0  
}
```

```
true.
```

```
?- new_value(thermo, temp, 15).  
true.
```

```
?- show_frame(ac).
```

```
Frame: ac {  
  temp:  
    Demons: readtd  
  cool: coolf  
  status: inactive  
  stop: stopf  
  warm: warmf  
}
```

```
true.
```

```
?- get_value(thermo,clima,C).  
C = comfort.
```

```
?- new_value(thermo, temp, 30).  
true.
```

```
?- show_frame(ac).
```

```
Frame: ac {  
  temp:  
    Demons: readtd  
  cool: coolf  
  status: cooling  
  stop: stopf  
  warm: warmf  
}
```

```
true.
```

```
?- get_value(thermo, clima, C).  
C = hot
```

```
?- new_value(thermo, temp, 48).  
true.
```

```
?- show_frame(ac).
```

```
Frame: ac {  
  temp:  
    Demons: readtd  
  cool: coolf  
  status: cooling  
  stop: stopf  
  warm: warmf  
}
```

```
true.
```

```
?- get_value(thermo, clima, C).  
C = burning.
```

```
?- get_value(ac, temp, T).  
T = 48.
```

→ *Collected from thermostat*

```
?- show_frame(alarm).
```

```
Frame: alarm {  
  date:  
  event:  
  temp:  
  count: 1  
}
```

```
true.
```

```
?- show_frame(alarm1).
```

```
Frame: alarm1 {  
  date: date(2020,5,5,12,46,7.632052421,0,UTC,-)  
  event: burning  
  is_a: alarm  
  temp: 48  
}
```

```
true.
```


We could, of course, add some I/O to the model

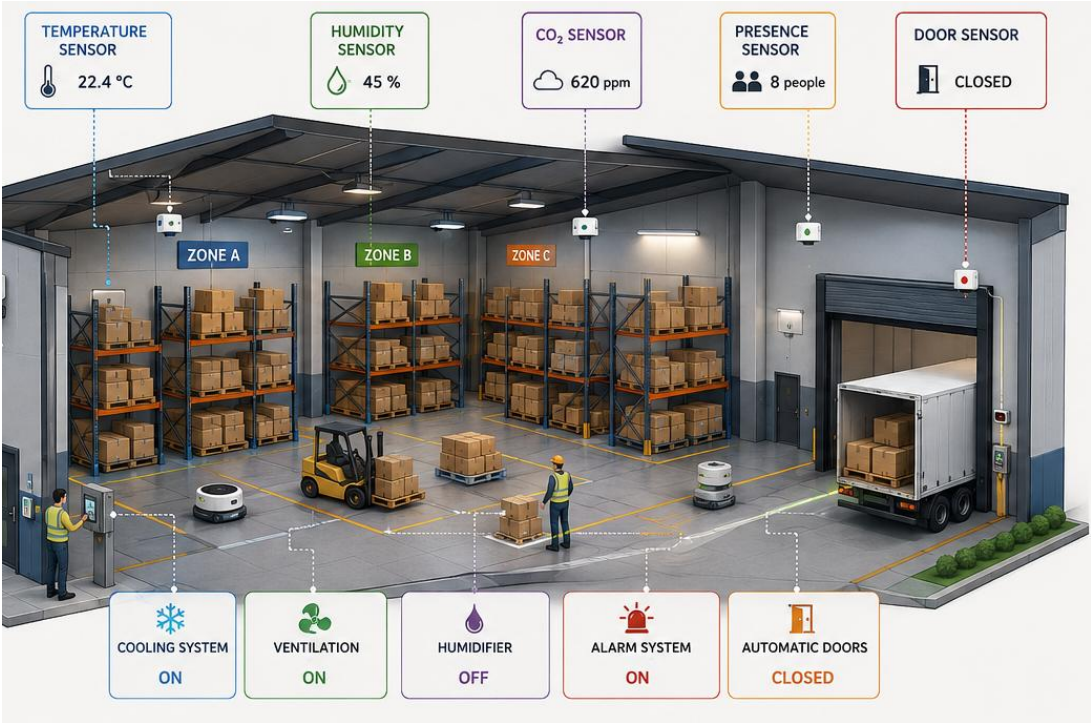
```
stopf(F) :- new_value(F, status, inactive), write('AC stopped'), nl.  
warmf(F) :- new_value(F, status, warming), write('AC is heating'), nl.  
coolf(F) :- new_value(F, status, cooling), write('AC is cooling'), nl.
```

...

```
genmsg(T, E, D) :- genname(N), new_frame(N), new_slot(N,is_a,alarm), new_value(N,event,E),  
                  new_value(N,temp,T), new_value(N,date,D), write('New alarm: '), write(N), nl.
```

...

Smart Warehouse



Use this example to continue with Labwork3.....